07

Linear Regression

The simplest model — drawing the best line through data

Predicting a number, and seeing how a model actually 'learns'

AI/ML Engineer Learning Series · Deep Dive Edition

What it does	Predicts a number (like price, marks, sales)
Type	Supervised learning — regression
The idea	Find the straight line that best fits the data
Why learn it	It's the foundation — every other model builds on this
Bonus	You'll finally see what 'training' really means inside

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**What is Linear Regression?**	The best-line idea
2	**The Line Equation**	y = mx + c, in plain words
3	**What 'Best' Means**	Errors and how we measure them
4	**The Cost Function**	One number that says how wrong we are
5	**How It Learns**	Gradient descent, step by step
6	**Multiple Features**	More than one input column
7	**Doing It in Code**	Full sklearn example
8	**Reading the Results**	What the numbers tell you
9	**Assumptions**	When linear regression works well
10	**Regularization**	Ridge and Lasso, kept simple
11	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. What is Linear Regression?

Linear Regression (■■■■■■■■ ■■■■■■■■) is the simplest machine learning model. Its job: look at your data and draw the straight line that fits it best. Once it has that line, it can predict new values.

Imagine you have data on study hours and exam marks. More hours usually means more marks. If you plot the points, they roughly follow an upward path. Linear regression finds the single best straight line through those points. Then, for any new number of study hours, you can read off the predicted marks from the line.

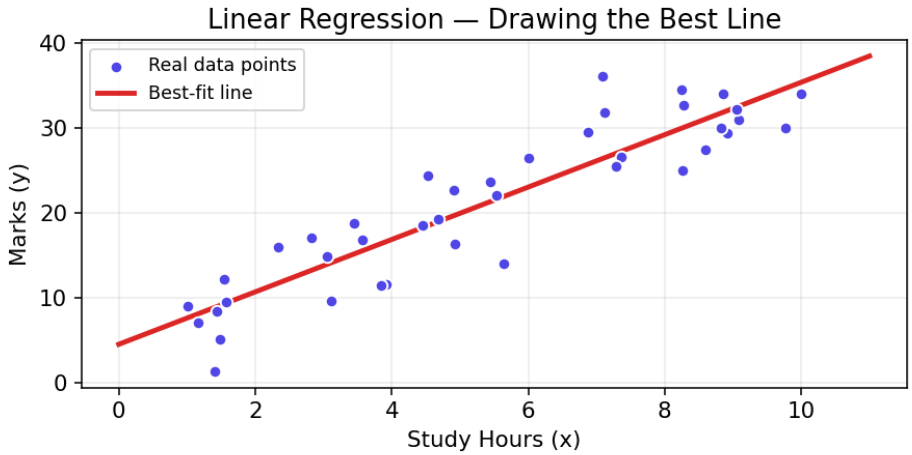

Fig 1 — The blue dots are real data. The red line is the best fit. That line is the 'model'.

'Linear' means straight line. 'Regression' means predicting a number (not a category). So linear regression = predicting a number using a straight line.

> ■ *It feels too simple to be useful, but it's everywhere — predicting prices, sales, temperatures, risk. And every advanced model is built on the same ideas you'll learn here.*

2. The Line Equation

Every straight line can be written with a simple formula you learned in school:

$$y = m \times x + c$$

In machine learning we use slightly different letters, but it's the same thing:

$$y = w \times x + b$$

Here's what each part means in plain words:

Symbol	Name	Plain meaning
y	prediction	The answer we want (e.g. marks)
x	feature / input	The thing we know (e.g. study hours)

w	weight / slope	How steep the line is — how much y changes when x grows
b	bias / intercept	Where the line starts (value of y when x = 0)

Training the model simply means finding the best values for **w** and **b** — the slope and starting point that make the line fit the data as closely as possible. That's it. The whole 'learning' is just adjusting these two numbers.

3. What Does 'Best' Mean? (Errors)

The line won't pass through every point perfectly. For each point there is a gap between the real value and what the line predicts. That gap is called the **error** or **residual** (■■■■■■■■■■ — ■■■■■■■■).

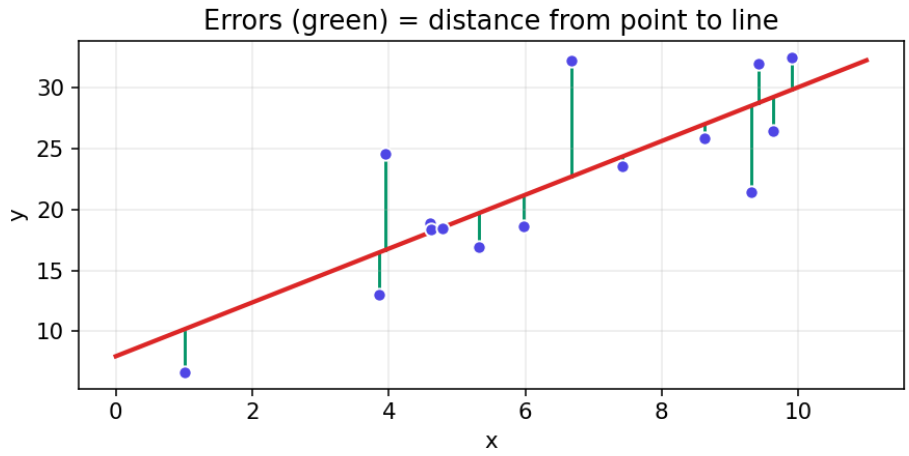

Fig 2 — Each green line is one error: the distance from a real point to the line.

The 'best' line is the one that makes all these errors as small as possible, all together. A small total error means the line fits the data well. A big total error means it fits poorly. So now we just need a way to add up all the errors into one number.

4. The Cost Function — One Number for 'How Wrong'

The **cost function** (■■■■ ■■■■■ — ■■■■■ ■■■■■■) turns all the errors into a single number. For linear regression, the usual one is **Mean Squared Error (MSE)**:

$$\text{MSE} = \text{average of } (\text{real} - \text{predicted})^2$$

Why square the errors? Two good reasons:

- Squaring makes every error positive, so errors above and below the line don't cancel out.
- Squaring punishes big mistakes much more than small ones, pushing the line to avoid large misses.

So the goal of training becomes very clear: **find the w and b that make the MSE as small as possible**. Lower cost = better line.

> ■ *MSE is in squared units, which can feel odd. That's why people often take its square root (called RMSE) to bring it back to normal units — like actual marks or actual taka.*

5. How It Learns — Gradient Descent

How does the model find the best w and b? It uses a clever method called **gradient descent** (■■■■■■■■■■■■ ■■■■■■■■). The idea is beautifully simple, like a ball rolling downhill to the lowest point.

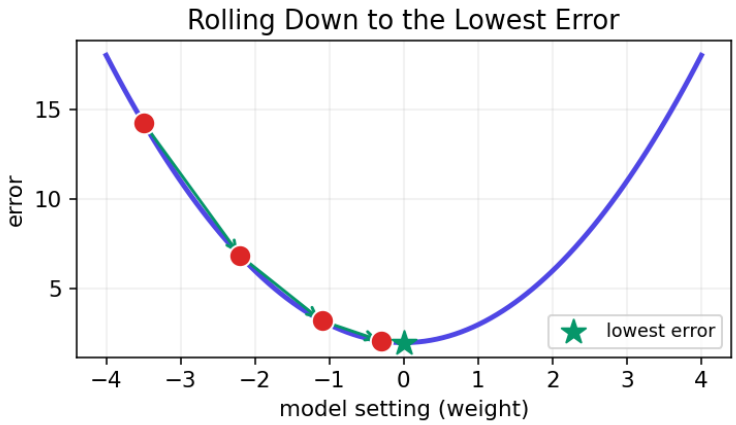

Fig 3 — The cost is a bowl shape. The ball (model) rolls down to the lowest error step by step.

Step by step, gradient descent does this:

- Start with random values for w and b (a random line).
- Measure the cost (how wrong the line is).
- Figure out which way to nudge w and b to make the cost a little smaller.
- Take a small step in that direction.
- Repeat hundreds of times until the cost stops dropping.

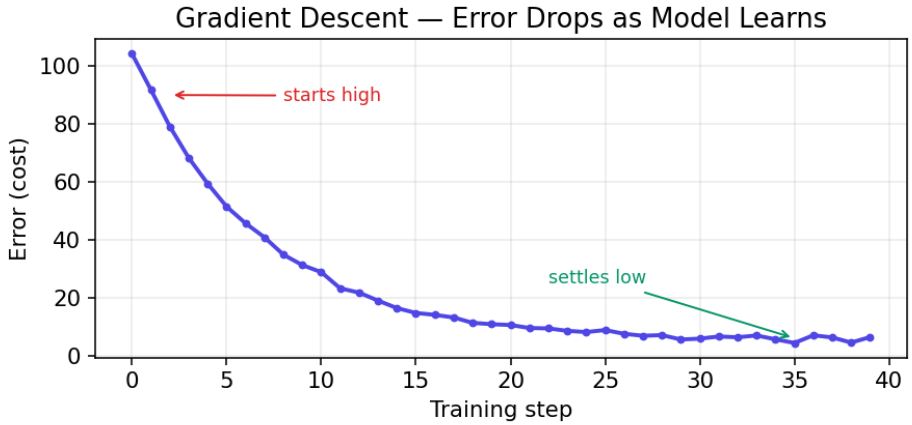

Fig 4 — As training steps go on, the error drops fast, then settles at its lowest point.

The size of each step is called the **learning rate** (■■■■■■■ ■■■). Too big and the ball jumps around and overshoots. Too small and it takes forever. Picking a good learning rate is one of the small arts of ML.

6. More Than One Input (Multiple Features)

Real problems use many inputs, not just one. To predict a house price you might use size, number of rooms, AND location. This is **multiple linear regression**. The idea is the same — just more weights, one per feature:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + ... + b$$

Each feature gets its own weight that says how important it is. A big weight means that feature strongly affects the prediction. A weight near zero means it barely matters. Instead of a line, the model now fits a flat sheet (or higher) through the data — but you never have to picture that; the math handles it.

■ *Because features can be on different scales (size in sq ft vs rooms 1-5), scaling them first (StandardScaler from the sklearn topic) helps the model learn fairly and faster.*

7. Doing It in Code

With sklearn, the whole thing is just a few lines — the fit/predict pattern again:

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# X = study hours (input), y = marks (answer)
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y = np.array([12, 25, 31, 44, 52, 61, 68, 82])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

# Create and train
model = LinearRegression()
model.fit(X_train, y_train)

# The learned line: y = w*x + b
print('slope (w):', model.coef_)        # e.g. [9.8]
print('intercept (b):', model.intercept_) # e.g. 2.1

# Predict
preds = model.predict(X_test)
print('R2 score:', r2_score(y_test, preds))

# Predict for a brand new value (10 study hours)
print(model.predict([[10]]))   # estimated marks
```

8. Reading the Results

After training, the model gives you these to look at:

Output	What it tells you
model.coef_	The weight(s) — how much each feature affects the answer
model.intercept_	The bias b — where the line starts
r2_score	How well the line fits, from 0 to 1 (closer to 1 is better)
MSE / RMSE	Average error size (smaller is better)

Understanding the weight: if the slope for study hours is 9.8, it means each extra hour of study adds about 9.8 marks. That's the beauty of linear regression — the results are easy to explain to anyone, even non-technical people.

What is a good R^2?

- **$R^2 = 1.0$** — perfect fit (rare in real life).
- **$R^2 = 0.7$-0.9** — usually a strong, useful model.
- **R^2 near 0** — the model barely explains anything; a straight line may be wrong here.

9. When Does It Work Well? (Assumptions)

Linear regression makes a few quiet assumptions (■■■■■■). When they hold, it works great. When they don't, the results can mislead you.

Assumption	Plain meaning
Linearity	The real relationship is roughly a straight line
Independence	Data points don't depend on each other
No extreme outliers	A few wild points can drag the whole line
Similar spread	Errors are about the same size across the range

■ *If your data curves instead of going straight, a straight line will fit poorly. In that case you either transform the data or use a more flexible model. Always draw a scatter plot first to check the shape.*

10. Regularization — Keeping the Model Humble

Sometimes a model leans too hard on certain features and overfits. **Regularization** (■■■■■■■■■■■■■■) gently pushes the weights toward smaller values, keeping the model simpler and more honest. Two popular versions:

Type	What it does	When to use
Ridge (L2)	Shrinks all weights a little	Many useful features
Lasso (L1)	Shrinks some weights to exactly zero	To drop useless features

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)   # alpha controls the strength
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)   # can zero-out weak features
lasso.fit(X_train, y_train)
```

■ *Lasso is handy for feature selection: features it sets to zero are ones it decided aren't worth keeping. A bigger alpha means stronger shrinking.*

11. Common Mistakes

Mistake	Why it's a problem	Fix
Using it on curved data	A line can't fit a curve	Check scatter plot first
Ignoring outliers	They drag the line off	Inspect and handle outliers
Not scaling features	Big-scale features dominate	Scale before training
Trusting a low R2	The line may be wrong shape	Try other models too
Reading cause from weights	Weight is link, not cause	Correlation is not cause

Quick Reference Cheatsheet

Task	Code
Import	from sklearn.linear_model import LinearRegression
Create model	model = LinearRegression()
Train	model.fit(X_train, y_train)
Predict	model.predict(X_test)
See slope (weights)	model.coef_
See intercept (bias)	model.intercept_
R2 score	r2_score(y_test, preds)
Mean squared error	mean_squared_error(y_test, preds)
RMSE	mean_squared_error(y_test, preds, squared=False)
Ridge regression	Ridge(alpha=1.0)
Lasso regression	Lasso(alpha=0.1)
Scale features	StandardScaler().fit_transform(X)

Key formulas to remember:

$$\text{Model: } y = w \cdot x + b \quad \text{Cost: MSE} = \text{avg}(\text{real} - \text{predicted})^2$$

Next Topic: Logistic Regression — despite the name, it's for classification (yes/no answers). We'll see how it bends the line into an S-shape to predict categories.